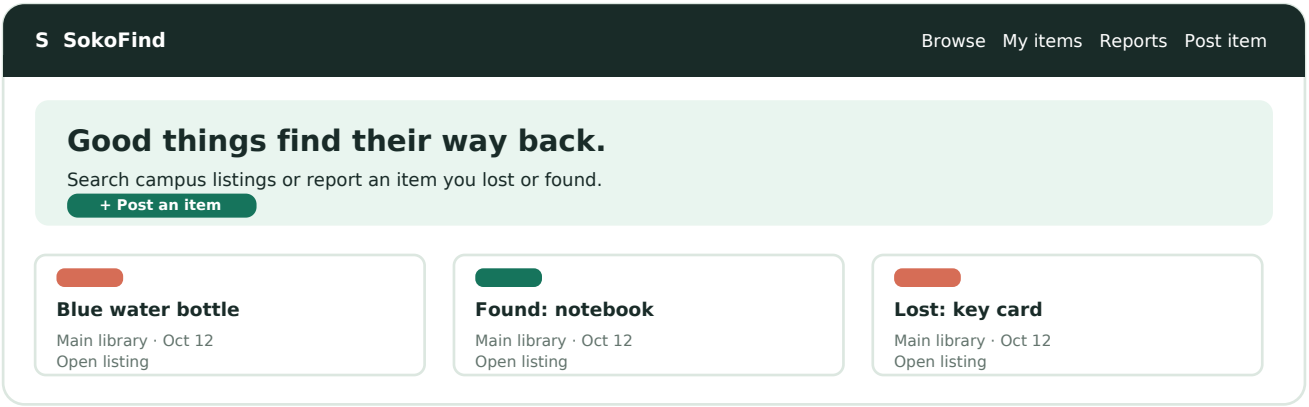
KIRINYAGA UNIVERSITY

SPI 2301 · WEB APPLICATION DEVELOPMENT II

SokoFind

Campus Lost & Found Web Application

SEMESTER PROJECT PROPOSAL, DESIGN & FINAL REPORT



Group 8 · Semester Project

Prepared for Kirinyaga University · SPI 2301

Group members and admission numbers are listed on the next page. Add the assigned lecturer and submission date if required by the department.

Project team

No.	Name	Admission number
1	Ndambuki James	CT100/G/25250/24
2	Derrick Macharia	CT100/G/25067/24
3	Daniel Gitere Nyaga	CT100/G/24732/24
4	Koech Kipruto Kevin	CT100/G/24920/24
5	Clinton Njoroge	CT100/G/23834/24
6	Njoroge Reginah Wairimu	CT100/G/23962/24
7	Regina Wanjiru Kamande	CT100/G/22896/24
8	Walter Odindo Abok	CT100/G/24104/24
9	Fredrick Mbithi	CT100/G/24477/24
10	Otieno Marvin Frank	CT100/G/22849/24

Abstract

SokoFind is a responsive campus lost-and-found application designed to make reports easier to publish, discover, and resolve. It provides student accounts, searchable lost and found listings, an owner-managed claim workflow, and three data reports. The solution uses PHP, MySQL, HTML, CSS, and JavaScript, with server-side validation, PDO prepared statements, session-based sign-in, and modular shared files.

Keywords: lost property, campus web application, PHP, MySQL, CRUD, reporting.

1. Project proposal

1.1 Project title

SokoFind: A Campus Lost & Found Web Application for Kirinyaga University

1.2 Problem statement

Students who lose belongings on campus often rely on informal class chats, noticeboards, or word of mouth. Reports may be fragmented, difficult to search, and quickly buried. A person who finds an item may not know where to report it, while an owner may not know which office or group has the information. Informal posts can also reveal sensitive details or expose personal contact information.

1.3 Problem justification

A shared, searchable board can make lost and found information easier to discover and keep in one place. Structured categories, locations, and dates improve search. A claim workflow lets a claimant describe a verification detail to the listing owner without publishing private contacts. Status tracking shows whether a listing is open, claimed, or returned, and reports summarize campus activity.

1.4 Objectives

General objective: To design and develop a dynamic, responsive web application that supports reporting, searching, claiming, and tracking lost and found property within the university community.

Specific objectives: (1) Provide account registration and secure sign-in for students. (2) Enable users to create, retrieve, update, and delete lost/found listings. (3) Provide search and filters by type and category. (4) Implement claim submission and owner review, with status updates. (5) Generate three reports describing category distribution, monthly activity, and listing outcomes. (6) Prepare the system for deployment on a PHP/MySQL host.

1.5 Scope

The first release covers student accounts, item records, claims, owner actions, and aggregate reports. It does not include email notifications, image uploads, administrator moderation, mobile apps, or verification against the university student registry. The app advises users to meet in public campus spaces and avoid sharing sensitive details.

1.6 Methodology

The group follows an iterative approach: gather and refine requirements; model actors, data, and flows; implement responsive views and PHP processing; connect MySQL through PDO; verify the required workflows; then deploy and document the hosted setup. The client renders HTML/CSS and provides browser validation; PHP performs authentication, authorization, validation, and database operations; MySQL persists users, items, and claims.

2. System analysis

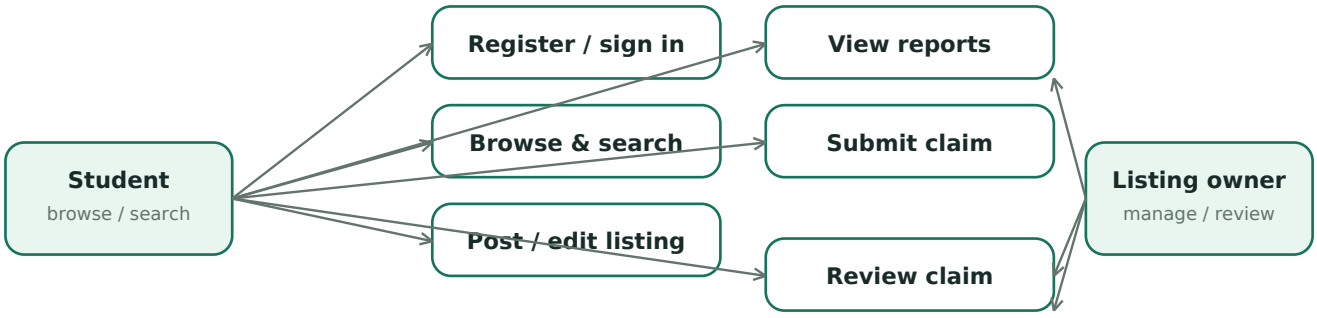
2.1 Functional requirements

ID	Requirement
FR-01	Visitors can browse open listings and filter by query, type, and category.
FR-02	Users can register, sign in, and sign out. Passwords are stored as hashes.
FR-03	Signed-in users can create, view, edit, and delete their own listings.
FR-04	A signed-in user can submit a claim message to another user’s open listing.
FR-05	Listing owners can accept or decline pending claims. Accepting a claim marks the item claimed and closes other pending claims.
FR-06	Owners can mark a listing returned, reopen it, and review claim history.
FR-07	Signed-in users can view category, monthly activity, and status/claim reports.

2.2 Non-functional requirements

Quality	Requirement
Security	Prepared SQL, escaped output, password hashing, CSRF tokens, session ID rotation, owner checks, and server validation.
Usability	Clear navigation, responsive layouts, meaningful error messages, and accessible labels.
Maintainability	Shared bootstrap/helpers and separated public pages, static assets, SQL schema, and documentation.
Reliability	Relational constraints, database indexes, unique account email, and unique claim per item/user.
Compatibility	PHP 8.1+, MySQL/MariaDB, and current mobile/desktop browsers.

2.3 UML use-case view

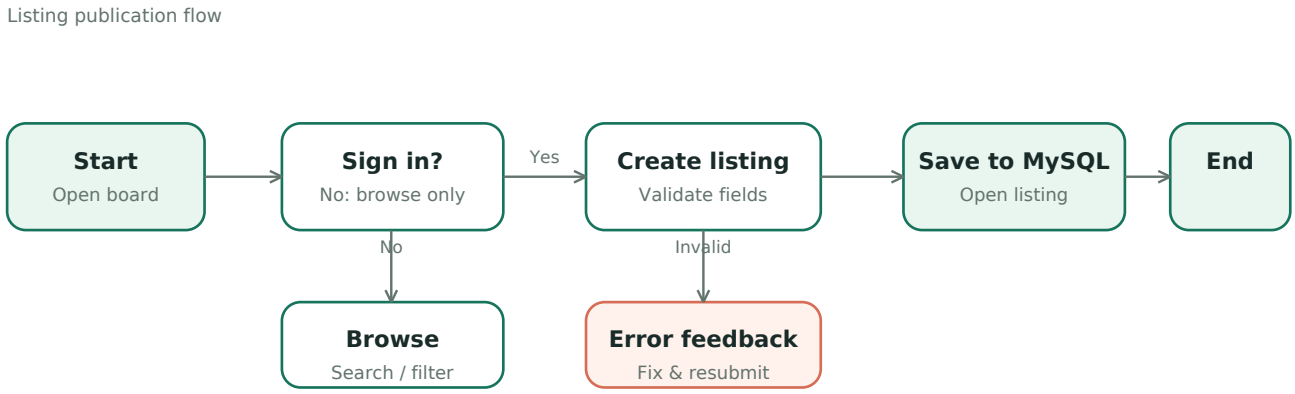


Actors use authenticated pages to create and manage listings; the owner alone can review claims.

The use-case model has two primary actors: a student and a listing owner. A student may also be a listing owner. Only an owner can edit/delete their listing or decide claims; sign-in is required for posting, claims, reports, and owner actions.

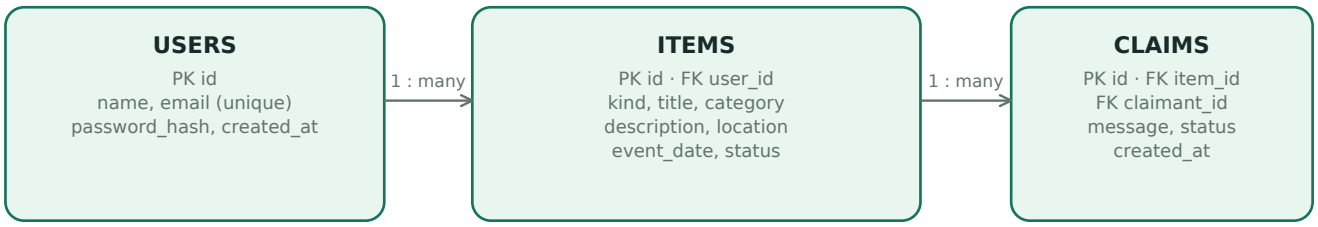
3. System design

3.1 Listing publication flow



Client-side constraints improve convenience; PHP repeats validation before writing to the database. Invalid input returns to the form with a message. Public browsing does not require an account.

3.2 Entity relationship diagram



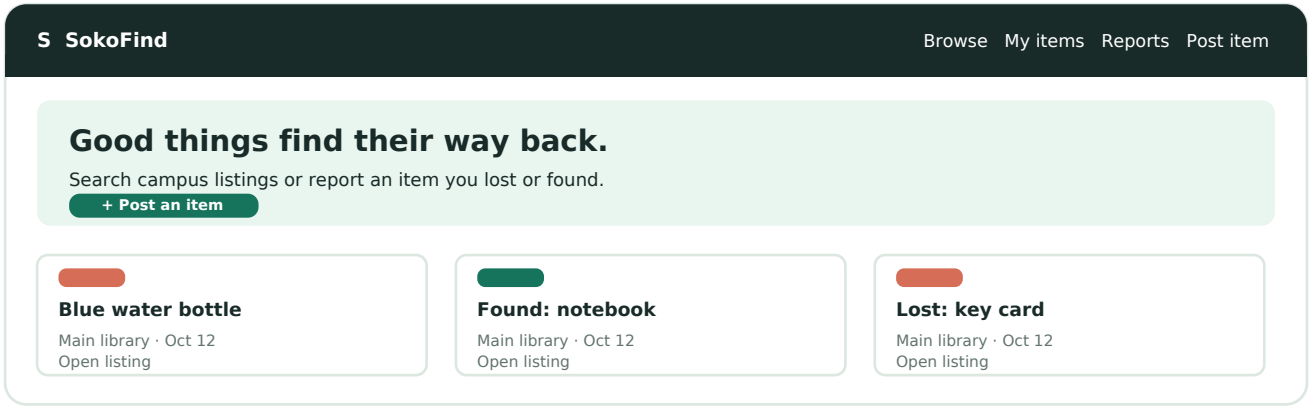
A user can own many items and submit many claims; an item can receive claims from other users.

Unique constraint: (item_id, claimant_id). Foreign keys cascade deletes to keep the board consistent.

Database schema

Table	Key fields	Relationship / rule
users	id, name, email, password_hash, created_at	Email is unique. One user owns many items and may submit many claims.
items	id, user_id, kind, title, category, description, location, event_date, status, timestamps	user_id references users.id; deletion cascades. Indexed for browsing and owner lists.
claims	id, item_id, claimant_id, message, status, created_at	item_id and claimant_id are foreign keys. One claim per item/user.

3.3 Interface wireframe



The implemented interface applies a green campus-community palette, responsive navigation, listing cards, search filters, status badges, and owner/claim controls. This design preview records the main board layout. Capture the live hosted interface after deployment.

4. Development and implementation

4.1 Technology and architecture

The client uses semantic HTML, responsive CSS, and small JavaScript helpers for navigation and confirmation prompts. PHP pages call shared helpers in app/bootstrap.php for session state, database access, output escaping, CSRF protection, redirects, and reusable layout. MySQL tables use foreign keys and indexes. PDO uses exception mode and native prepared statements.

4.2 Project structure

Path	Purpose
public/	Public entry pages and styles/scripts. Configure this as the web document root.
app/	Database/session helpers, item card, and private local DB configuration. Keep outside the public web root.
sql/schema.sql	MySQL schema and relationships.
docs/	Project report and submission materials.

4.3 Database operations and reports

CRUD behavior covers listing creation, retrieval, editing, and deletion. Claim records can be created and reviewed. The reports use grouped aggregate queries: listing totals by category; lost/found totals by month in the latest six months; and listing status totals with claim counts. The report page derives proportional bar widths from the query results.

4.4 Security and validation

Passwords use PHP password_hash/password_verify. Mutating forms carry CSRF tokens. Database writes use prepared statements. Text output is HTML-escaped. Ownership is enforced in update/delete queries and claim-review actions. Required fields, allowed categories/statuses, date ranges, and text lengths are checked both in the browser and on the server.

4.5 Development walkthrough

Create the schema; configure the database connection; start the PHP built-in server using public/ as document root; register two users; add and search listings; submit and review a claim; inspect the reports. Deployment instructions are in README.md. External credentials stay out of the repository.

5. Testing

The following acceptance test cases map to the key project requirements. Record the execution date, tester initials, actual result, and pass/fail result during the group demonstration. The repository includes a Docker environment with PHP 8.3 and MySQL 8.4. Run these cases with **docker compose up -d --build** and record the execution date, tester initials, and pass/fail result during the group demonstration.

ID	Test / steps	Expected result
TC-01	Register with a valid name, email, and password of at least 8 characters.	Account is created; user is signed in and redirected to the board.
TC-02	Submit a listing with a missing title, invalid category, or future date.	Server rejects the entry and displays validation feedback; no invalid row is stored.
TC-03	Search for an open item; apply type and category filters.	Only matching open listings appear; text is escaped safely.
TC-04	A second user claims an open found item; owner accepts the request.	Claim is stored and reviewable; item becomes Claimed; other pending claims close.
TC-05	Edit and delete one’s own listing; attempt to change another user’s listing URL.	Owner actions succeed; unauthorized modification is denied/not found.
TC-06	Open Reports after sample listings and claims are entered.	Category, monthly, listing status, and claim totals match the database rows.

6. User manual

Create an account: choose Create account, enter a name, email, and password, then submit. Browse: search by item, description, or location and filter by lost/found type or category. Post: sign in, choose Post an item, enter the item type, category, name, description, location, and date. Avoid posting private identifiers.

Manage: My items shows your own listings. Edit details, mark an item returned, reopen it, or delete it. Claim: sign in to send a short message with a detail that helps verify ownership. The listing owner opens the listing and accepts or declines. Reports: sign in and choose Reports to see the three aggregate summaries. Sign out: use Log out in the navigation.

Safety: arrange handover in a public campus location; do not post passwords, complete identity numbers, or private contact details.

7. Deployment guide

Source repository: https://github.com/FredrickMbithi/SokoFind. Create a MySQL database at a host with supported PHP and HTTPS. Import sql/schema.sql, deploy the files, set the web root to public/, then create app/config.local.php on the server with database settings. If the host only provides public_html, upload the public/ contents there and keep app/ one directory above it. Test registration, posting, claiming, owner review, and reports on the live site.